

Time Series Forecasting

In many real-life situations, data is not collected at a single point in time but is recorded continuously or periodically over a span of time. Such data helps us understand how a phenomenon changes, grows, or fluctuates as time passes. This type of data is known as time series data.

A time series consists of observations arranged in chronological order. The defining feature of time series data is that time plays a central role in analysis. Unlike ordinary datasets, the sequence in which observations occur cannot be ignored or rearranged because each value may be influenced by earlier values.

Examples:

Situation	Time Series Variable
Daily temperature	Temperature vs Day
Website traffic	Visitors per day
Stock market	Share price vs time
School attendance	Students per day
Electricity usage	Consumption per hour

Need for Time Series forecasting

Time series forecasting is required to predict future values based on past observations collected over time. Since many real-world phenomena such as sales, population growth, weather conditions, and website traffic change continuously, analysing past patterns helps in estimating future trends.

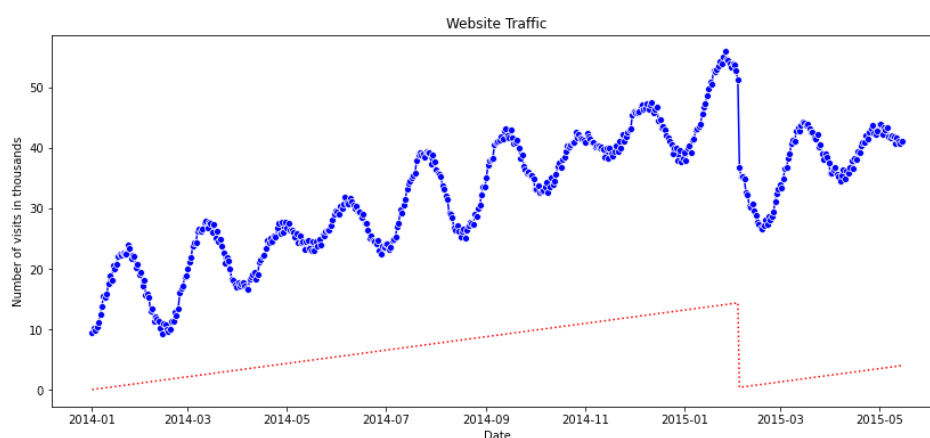
Forecasting enables organizations and individuals to make informed decisions, plan resources efficiently, and reduce uncertainty. Businesses use time series forecasting to estimate future demand and manage production, governments apply it for economic planning, and scientists use it to predict environmental and climatic changes.

Thus, time series forecasting is essential for effective planning, better decision-making, and anticipating future outcomes based on historical data.

Components of Time Series

For analytical purposes, a time series is often decomposed into several components. Breaking a time series into components helps us understand the underlying structure of the data and interpret patterns more clearly. A time series may generally be divided into level, trend, seasonality, cyclicity, and noise.

Consider the website traffic data shown in the given image, where the number of website visits is recorded over different dates.



1. **Level:** The level represents the average or baseline value around which the data fluctuates over time. It indicates the general magnitude of the series without considering upward or downward movement.

Example: In the website traffic graph, most values lie around a certain range of visits (for instance, around the middle of the vertical scale). This average number of visitors forms the level of the time series.

2. **Trend:** The trend shows the long-term direction of movement in the data, whether increasing, decreasing, or stable over time.

Example: In the image, website traffic gradually increases as time progresses. This overall upward movement indicates a positive trend, suggesting growing popularity of the website.

3. **Seasonality:** Seasonality refers to regular and repeating patterns occurring at fixed and known intervals of time, such as daily, weekly, or yearly cycles.

Example: If the website consistently receives higher traffic on certain days (for example weekends) and lower traffic on others, the repeating rise and fall represent seasonal variation.

4. **Cyclicity:** Cyclicity describes long-term fluctuations that occur over extended periods but do not follow a fixed or regular interval. These movements are often influenced by broader economic or social factors.

Example: If website traffic increases during periods of high online activity and decreases during slower economic periods over several months or years, these long-duration waves represent cyclical behaviour.

5. **Noise (Irregular Component):** Noise consists of random and unpredictable variations that cannot be explained by trend, seasonality, or cycles. These changes occur due to unexpected events.

Example: A sudden drop in website visits caused by a temporary server failure or an unexpected spike due to viral content represents noise in the time series.

Stationarity in Time Series

In time series analysis, the concept of stationarity plays an important role in understanding and modelling data observed over time. A time series is said to be stationary when its statistical properties remain constant throughout the period of observation. In other words, the behaviour of the series does not change with time.

For a time series to be stationary, its mean, variance, and overall pattern of fluctuations should remain stable. The data may vary from one time point to another, but these variations occur around a constant average level and with approximately the same degree of dispersion. A stationary series therefore does not exhibit long-term growth, decline, or changing variability.

In contrast, many real-world time series are non-stationary, meaning their statistical characteristics change over time due to trends, seasonal effects, or external influences.

Example:

Supermarket Sales: Consider the daily sales of a supermarket recorded over a period of time. If the supermarket becomes increasingly popular and sales steadily rise month after month, the average level of sales changes over time. Similarly, during festive seasons, sales may increase sharply, producing large fluctuations. Since both the mean and variability are

changing, such a series is non-stationary.

On the other hand, suppose the supermarket records nearly the same amount of sales each day, with only small random variations around an average value. The sales neither show a continuous increase nor a persistent decrease, and the spread of observations remains approximately constant. In this situation, the time series is stationary because its statistical properties remain unchanged over time.

Additive and Multiplicative Time Series

In time series analysis, an observed series is usually considered to be composed of several underlying components, namely trend, seasonal variation, and irregular variation. These components combine in different ways to produce the observed values of the series. Depending on the manner in which these components interact, time series models are broadly classified into additive and multiplicative models.

Additive Time Series

An additive time series is one in which the components combine by addition. The observed value at any time is obtained by adding the effects of the trend, seasonal, and irregular components.

The additive model may be expressed as:

$$Y = T + S + I$$

where,

- Y represents the observed value
- T is the trend component
- S is the seasonal component
- I is the irregular component

The additive model is appropriate when seasonal fluctuations remain approximately constant over time, regardless of the level of the series. In such cases, the magnitude of variation does not increase or decrease as the trend changes.

For example, consider supermarket sales where festive-season sales increase by nearly the same amount each year. If sales rise by about ₹10,000 every festival season irrespective of overall growth in business, the seasonal effect is constant, and the additive model suitably represents the series.

Multiplicative Time Series

A multiplicative time series is one in which the components combine through multiplication. The observed value is obtained by multiplying the trend, seasonal, and irregular components.

The multiplicative model is written as:

$$Y = T \times S \times I$$

In this model, seasonal variations depend on the level of the time series. As the trend increases, the magnitude of fluctuations also increases proportionally. Thus, variations are expressed in relative or percentage terms rather than absolute amounts.

For instance, if supermarket sales during festivals are consistently about 20 percent higher than normal sales, then the seasonal effect grows as overall sales increase. When average sales double, the seasonal increase also doubles, indicating a multiplicative relationship.

Evaluation of Forecast Accuracy Using RMSE and MAPE

Consider the simplified web traffic data representing the number of website visits (in thousands) recorded over ten months.

Month	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov
Number of Visits in Thousands	26	8	17	29	34	17	22	19	16	22

To evaluate the performance of a forecasting model, the dataset is divided into two parts: a **training set** and a **validation set**. The first seven observations (February to August) are used to build the model, while the last three observations (September to November) are used to test forecast accuracy.

Training and Validation Sets

Training data:

26, 8, 17, 29, 34, 17, 22

Validation data (actual values):

September = 19, October = 16, November = 22

A simple naïve forecasting method is applied in which the last observed value from the training data is taken as the forecast for future periods. Since the last training observation (August) equals 22, the forecasts for September, October, and November are all taken as 22.

Root Mean Square Error (RMSE)

The Root Mean Square Error measures the average magnitude of forecasting errors by giving greater weight to larger deviations. It is defined as the square root of the mean of squared forecast errors.

$$RMSE = \sqrt{\frac{\sum (F - A)^2}{n}}$$

where,

- F denotes forecast values
- A denotes actual values
- n is the number of validation observations

The squared errors are:

- September: $(22 - 19)^2 = 9$
- October: $(22 - 16)^2 = 36$
- November: $(22 - 22)^2 = 0$

$$RMSE = \sqrt{\frac{9+36+0}{3}} = \sqrt{15} \approx 3.87$$

Thus, the average forecasting error is approximately **3.87 thousand visits**.

Mean Absolute Percentage Error (MAPE)

The Mean Absolute Percentage Error expresses forecast accuracy in percentage terms, making it easier to interpret relative error.

$$MAPE = \frac{1}{n} \sum \left| \frac{A - F}{A} \right| \times 100$$

The percentage errors are:

- September: $\frac{3}{19} = 0.1579$

- October: $\frac{6}{16} = 0.3750$
- November: 0

$$MAPE = \frac{0.1579 + 0.3750 + 0}{3} \times 100 \approx 17.76\%$$

Hence, the average forecasting error equals **17.76 percent** of the actual values.

RMSE provides the error magnitude in the same unit as the data, while MAPE expresses the error as a percentage, allowing comparison across datasets. In this example, the forecasting model produces moderate accuracy, with an average deviation of about 3.87 thousand visits and an average percentage error of approximately 17.76 percent.

These measures are commonly used to assess and compare forecasting models in time series analysis.

Simple Models

Linear Regression

Consider the web traffic data (number of visits in thousands) shown in the table. The first seven observations are used as **training data**, while the last three observations are reserved for **validation** of the model.

Month	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov
Number of Visits in Thousands (y)	26	8	17	29	34	17	22	19	16	22
x	0	1	2	3	4	5	6	7	8	9
Predictions	19.39	20.21	21.04	21.86	22.68	23.5	24.32	25.14	25.96	26.79

Linear Regression Model

A linear regression model assumes a linear relationship between time (x) and visits (y):

$$y = a + bx$$

where,

- a = intercept

- b = slope of the regression line

Using the **training data (x = 0 to 6)**, the estimated regression equation is:

$$y = 19.39 + 0.82x$$

This equation represents the fitted regression line for website traffic.

Model Evaluation

The forecasting performance is evaluated using Root Mean Square Error (RMSE) and Mean Absolute Percentage Error (MAPE) based on validation data (Sep–Nov).

Validation Data

Month	Actual	Forecast
Sep	19	25.14
Oct	16	25.96
Nov	22	26.79

1. Root Mean Square Error (RMSE)

$$RMSE = \sqrt{\frac{\sum (F - A)^2}{n}}$$

$$RMSE \approx 7.30$$

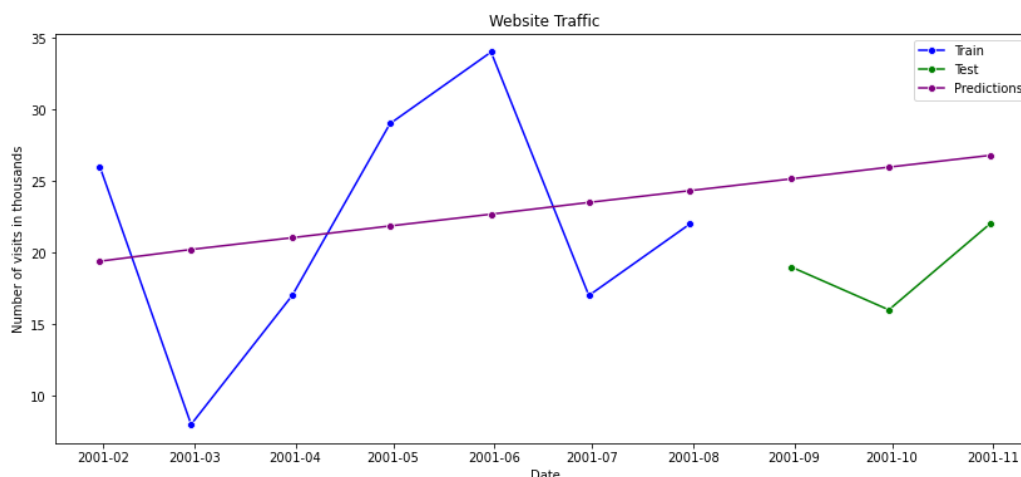
Thus, the average forecasting error is about **7.30 thousand visits**.

2. Mean Absolute Percentage Error (MAPE)

$$MAPE = \frac{1}{n} \sum \left| \frac{A - F}{A} \right| \times 100$$

$$MAPE \approx 38.79\%$$

Hence, the average percentage forecasting error is approximately **38.79%**.



The fitted regression line shows a slight upward trend in website traffic over time. However, the relatively high RMSE and MAPE values indicate that the linear regression model does not capture the fluctuations in the data very well. The web traffic series contains irregular variations that are not fully explained by a simple linear trend.

Therefore, while linear regression provides a basic trend estimate, more advanced time series models may be required for improved forecasting accuracy.

Naïve Model

The naïve forecasting model is one of the simplest methods used in time series analysis. It assumes that the best prediction for the next time period is the most recent observed value. In other words, the future value is taken to be equal to the last available observation.

Mathematically, the naïve forecast is expressed as:

$$\hat{y}_{t+1} = y_t$$

where,

- y_t is the latest observed value
- $\hat{y}_{t+1}$ is the forecast for the next period

Model Evaluation

Forecast accuracy is measured using Root Mean Square Error (RMSE) and Mean Absolute Percentage Error (MAPE).

1. Root Mean Square Error (RMSE)

RMSE measures the average magnitude of forecasting errors.

$$RMSE = \sqrt{\frac{\sum (F - A)^2}{n}}$$

Squared errors:

- September: $(22 - 19)^2 = 9$
- October: $(22 - 16)^2 = 36$
- November: $(22 - 22)^2 = 0$

$$RMSE = \sqrt{\frac{9 + 36 + 0}{3}} = \sqrt{15} \approx 3.87$$

Thus, the average prediction error equals **3.87 thousand visits**.

2. Mean Absolute Percentage Error (MAPE)

MAPE expresses forecast error as a percentage.

$$MAPE = \frac{1}{n} \sum \left| \frac{A-F}{A} \right| \times 100$$

Percentage errors:

- September: $3/19 = 15.79\%$
- October: $6/16 = 37.50\%$
- November: 0%

$$MAPE = \frac{15.79 + 37.50 + 0}{3} \approx 17.76\%$$

Hence, the average percentage error is **17.76%**.

Interpretation

The naïve model assumes no underlying trend or pattern and simply carries forward the most recent observation. Despite its simplicity, the model produces lower error measures compared to the linear regression model for this dataset. This suggests that the web traffic data does not follow a strong linear trend and is better approximated by recent observations.

The naïve model is therefore often used as a **baseline forecasting method**, against which

more complex models are compared.

Simple Average Method

The simple average method forecasts future values using the average of all past observations in the training dataset. It assumes that future values will be close to the historical mean.

$$\hat{y} = \frac{\sum y}{n}$$

Compute Average of Training Data

Training values:

26, 8, 17, 29, 34, 17, 22

$$\text{Average} = \frac{26+8+17+29+34+17+22}{7} = \frac{153}{7} = 21.86$$

Forecast Values

The same average is used for all validation periods.

Month	Actual (A)	Forecast (F)
Sep	19	21.86
Oct	16	21.86
Nov	22	21.86

Performance Measures

1. RMSE

Squared errors:

- Sep: $(21.86 - 19)^2 = 8.18$
- Oct: $(21.86 - 16)^2 = 34.34$
- Nov: $(21.86 - 22)^2 = 0.02$

$$RMSE = \sqrt{\frac{8.18+34.34+0.02}{3}} = \sqrt{14.18} \approx 3.77$$

2. MAPE

$$MAPE = \frac{1}{3} \left(\frac{2.86}{19} + \frac{5.86}{16} + \frac{0.14}{22} \right) \times 100$$

$$MAPE \approx 17.43\%$$

Simple Moving Average Method

The **simple moving average (SMA)** method forecasts values using the average of the most recent k observations. It gives more importance to recent data than older data.

Here, a **3-period moving average** is used.

Compute Moving Average Forecasts

Last three training observations (Jun–Aug):

34, 17, 22

$$\text{Forecast for Sep} = \frac{34 + 17 + 22}{3} = 24.33$$

For multi-step forecasting, the same moving average is used (since future actual values are unknown).

Forecast Values

Month	Actual (A)	Forecast (F)
Sep	19	24.33
Oct	16	24.33
Nov	22	24.33

Performance Measures

1. RMSE

Squared errors:

- Sep: $5.33^2 = 28.41$
- Oct: $8.33^2 = 69.39$

- Nov: $2.33^2 = 5.43$

$$RMSE = \sqrt{\frac{28.41 + 69.39 + 5.43}{3}} = \sqrt{34.41} \approx 5.87$$

2. MAPE

$$MAPE = \frac{1}{3} \left(\frac{5.33}{19} + \frac{8.33}{16} + \frac{2.33}{22} \right) \times 100$$

$$MAPE \approx 30.25\%$$

Interpretation

The simple average method provides stable forecasts based on the overall historical level of the data and performs reasonably well for this dataset. The simple moving average, while emphasizing recent observations, produces larger errors because recent values were relatively high compared to the validation data.

Thus, in this example:

- Simple Average performs better than Moving Average.
- Both methods serve as basic forecasting techniques and benchmarks for more advanced models.

Exponential Smoothing Models

Exponential smoothing models are a class of forecasting techniques used in time series analysis in which past observations are weighted using exponentially decreasing weights. Recent observations receive greater importance, while older observations receive progressively smaller weights. This property makes exponential smoothing suitable for data where recent behaviour is more relevant for forecasting than distant past values.

Unlike simple averages or moving averages, exponential smoothing updates forecasts continuously as new observations become available. Depending on the characteristics of the time series, different forms of exponential smoothing are used.

Exponential smoothing models are broadly classified into:

1. **Simple Exponential Smoothing (SES):** for data with no trend or seasonality.
2. **Holt's Exponential Smoothing:** for data with trend.
3. **Holt-Winters Exponential Smoothing:** for data with both trend and seasonality.

Simple Exponential Smoothing Technique (SEST)

Simple Exponential Smoothing is used when a time series fluctuates around a constant level without trend or seasonal variation. The forecast is calculated as a weighted average of the previous forecast and the latest observation.

The updating equation is:

$$F_{t+1} = \alpha Y_t + (1 - \alpha)F_t$$

where,

- Y_t = actual value at time t
- F_t = forecast at time t
- α = smoothing constant ($0 < \alpha < 1$)

A larger value of α gives more importance to recent data.

Application (Example: Web Traffic Data)

Visits (in thousands):

26, 8, 17, 29, 34, 17, 22

Assume:

- Initial forecast $F_1 = 26$
- Smoothing constant $\alpha = 0.3$

Forecast calculations:

$$\begin{aligned} F_2 &= 26 \\ F_3 &= 0.3(8) + 0.7(26) = 20.6 \\ F_4 &= 0.3(17) + 0.7(20.6) = 19.52 \\ F_5 &= 0.3(29) + 0.7(19.52) = 22.36 \\ F_6 &= 0.3(34) + 0.7(22.36) = 25.85 \\ F_7 &= 0.3(17) + 0.7(25.85) = 23.20 \\ F_8 &= 0.3(22) + 0.7(23.20) = 22.84 \end{aligned}$$

Thus, the forecast for the next period is approximately **22.84 thousand visits**.

When SES is Used

- No trend
- No seasonality
- Short-term forecasting

Holt's Exponential Smoothing (Double Exponential Smoothing)

Holt's method extends simple exponential smoothing by incorporating a **trend component**. It estimates both the level and the trend of the series.

Two equations are used:

Level equation

$$L_t = \alpha Y_t + (1 - \alpha)(L_{t-1} + T_{t-1})$$

Trend equation

$$T_t = \beta(L_t - L_{t-1}) + (1 - \beta)T_{t-1}$$

Forecast equation

$$F_{t+m} = L_t + mT_t$$

where,

- L_t = level component
- T_t = trend component
- α, β = smoothing parameters

Note:

- The level captures the current average value.
- The trend measures the direction and rate of change.
- Forecasts increase or decrease according to the estimated trend.

Holt's method is suitable for time series showing a **consistent upward or downward movement**, such as increasing website traffic or growing sales.

Holt–Winters Exponential Smoothing

Holt–Winters method further extends Holt's model by incorporating **seasonality** along with level and trend components. It is used when data exhibits repeating seasonal patterns.

There are two forms:

- **Holt–Winters** (constant seasonal effect)
- **Multiplicative Holt–Winters** (proportional seasonal effect)

Additive Holt–Winters Model

$$\begin{aligned}
 L_t &= \alpha(Y_t - S_{t-s}) + (1 - \alpha)(L_{t-1} + T_{t-1}) \\
 T_t &= \beta(L_t - L_{t-1}) + (1 - \beta)T_{t-1} \\
 S_t &= \gamma(Y_t - L_t) + (1 - \gamma)S_{t-s} \\
 F_{t+m} &= L_t + mT_t + S_{t-s+m}
 \end{aligned}$$

where,

- S_t = seasonal component
- s = seasonal length

Note:

- Level represents the base value.
- Trend represents long-term movement.
- Seasonal component captures repeating cycles such as monthly or quarterly patterns.

This method is widely used for forecasting retail sales, electricity demand, airline passengers, and website traffic with seasonal behaviour.

Summary

Method	Components Considered	Suitable Data
Simple Exponential Smoothing	Level only	No trend, no seasonality
Holt's Method	Level + Trend	Trending data
Holt–Winters Method	Level + Trend + Seasonality	Seasonal data

Exponential smoothing models provide flexible and efficient forecasting techniques by assigning greater importance to recent observations while still incorporating past information. They are therefore widely applied in practical time series forecasting problems.

Determination of Parameters in Exponential Smoothing Models

Exponential smoothing methods depend on certain smoothing parameters that control how much importance is given to recent observations compared to past data. Proper selection of these parameters is essential because they directly influence forecasting accuracy.

In exponential smoothing models, parameters determine the rate at which old information decays and new information is incorporated into forecasts.

Depending on the model used, different parameters are required:

Model	Parameters	Purpose
Simple Exponential Smoothing	α	Controls level updating
Holt's Method	α, β	Controls level and trend
Holt's - Winter Method	α, β, γ	Controls level, trend, and seasonality

Each parameter lies between 0 and 1.

Error Minimization (Most Common Method)

Parameters are chosen by minimizing a forecasting error measure such as:

- Mean Squared Error (MSE)
- Mean Absolute Error (MAE)
- Mean Absolute Percentage Error (MAPE)

The objective is to choose parameters that minimize forecast error. Modern statistical software automatically performs this optimization.

Determination of Parameters through Numerical Optimization

In exponential smoothing models, the smoothing parameters are selected so that the forecasting model produces the minimum possible error. This process is known as numerical optimization.

Instead of choosing parameters manually, mathematical algorithms automatically search for the best values.

Optimization Objective Function

Let

- y_t = actual value at time t
- $\hat{y}_t$ = forecast value
- $e_t = y_t - \hat{y}_t$ = forecast error

The optimization algorithm minimizes an error function such as:

- Mean Squared Error (MSE)

- Root Mean Squared Error (RMSE)
- Mean Absolute Percentage Error (MAPE)

Most commonly, **Sum of Squared Errors (SSE)** is minimized:

$$SSE = \sum_{t=1}^n (y_t - \hat{y}_t)^2$$

The parameters are chosen so that SSE is minimum.

Numerical Optimization in Simple Exponential Smoothing

In Simple Exponential Smoothing, the forecasting equation is:

$$\hat{y}_{t+1} = \alpha y_t + (1 - \alpha) \hat{y}_t$$

Here, the unknown parameter is α .

Optimization Steps

1. Choose an initial value of α (for example 0.1).
2. Compute forecasts using the smoothing equation.
3. Calculate forecast errors.
4. Compute SSE (or RMSE/MAPE).
5. Change α slightly.
6. Repeat calculations.
7. Select the α giving minimum error.

This iterative adjustment continues automatically until improvement stops.

4. Optimization in Holt's Method

Holt's exponential smoothing includes two parameters:

- $\alpha \rightarrow$ level smoothing
- $\beta \rightarrow$ trend smoothing

The optimization problem becomes:

$$\min_{\alpha, \beta} SSE(\alpha, \beta)$$

The algorithm searches combinations of α and β between 0 and 1 and identifies the pair producing the lowest forecasting error.

5. Optimization in Holt–Winters Method

For seasonal data, three parameters must be estimated:

- α (level)
- β (trend)
- γ (seasonality)

The optimization becomes:

$$\min_{\alpha, \beta, \gamma} SSE(\alpha, \beta, \gamma)$$

Because testing all combinations manually is impossible, numerical algorithms are used.

Error–Trend–Seasonal (ETS) Models

Error–Trend–Seasonal (ETS) models form a general framework for time series forecasting based on exponential smoothing methods. These models describe a time series by explicitly combining three fundamental components: error, trend, and seasonality.

The ETS framework provides a systematic way to represent different exponential smoothing models within a single structure.

Basic Idea of ETS Models

A time series y_t is assumed to consist of:

- **Error (E):** Random variation or irregular fluctuations.
- **Trend (T):** Long-term direction of movement.
- **Seasonality (S):** Regular repeating pattern over fixed intervals.

Thus, an ETS model explains how these three components interact to generate observed data.

Components of ETS Models

- Error Component (E):** The error represents unpredictable variations that cannot be explained by trend or seasonality. Two types exist:
 - **Additive Error (A):** Variations remain roughly constant.

- **Multiplicative Error (M):** Variations increase with the level of the series.
- b) **Trend Component (T):** The trend captures long-term movement in the data. Three types exist:
- **None (N):** No long-term increase or decrease.
 - **Additive Trend (A):** Linear increase or decrease.
 - **Damped Trend (Ad):** Trend gradually slows over time.
- c) **Seasonal Component (S):** Seasonality represents recurring patterns such as monthly or quarterly cycles. Three types exist:
- **None (N):** No seasonal pattern.
 - **Additive Seasonality (A):** Seasonal effect remains constant.
 - **Multiplicative Seasonality (M):** Seasonal effect changes with level.

ETS Model Notation

ETS models are written in the form:

$$ETS(E, T, S)$$

where:

- E = Error type
- T = Trend type
- S = Seasonal type

Examples

- **ETS(A,N,N)** → Simple Exponential Smoothing
- **ETS(A,A,N)** → Holt's Trend Model
- **ETS(A,A,A)** → Holt-Winters Additive Model
- **ETS(M,A,M)** → Multiplicative seasonal model

The ETS framework helps analysts choose a forecasting model depending on data characteristics:

- Stable data → No trend or seasonality
- Increasing data → Trend component needed
- Periodic fluctuations → Seasonal component required

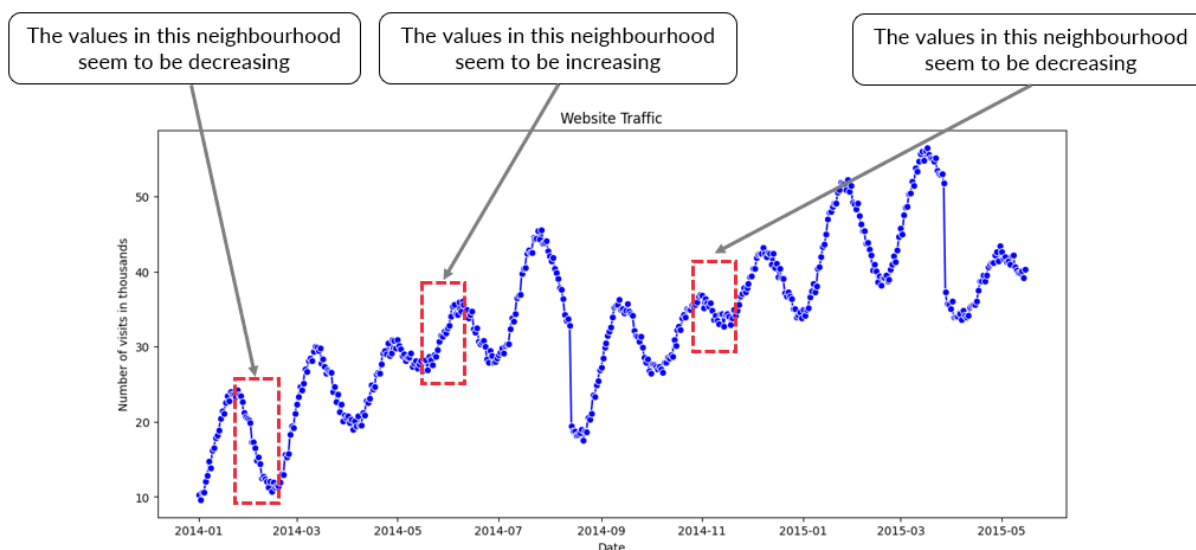
ETS models are widely used because they:

- Adapt automatically to changing patterns
- Provide accurate short-term forecasts
- Combine statistical theory with practical forecasting
- Form the basis of many modern forecasting systems

Forecasting Techniques Based on Autocorrelation

In addition to smoothing and decomposition methods, another important group of forecasting techniques is based on the concept of **autocorrelation**. Autocorrelation measures the relationship between a time series and its own past values.

In many time-dependent datasets, current observations are influenced by previous observations. For example, today's sales or website traffic are often related to the values observed in earlier periods. Autocorrelation-based models use this internal dependence to predict future values.



Unlike exponential smoothing methods, which focus on averaging past data, these techniques analyse how strongly observations are correlated across different time lags. By modelling these relationships mathematically, forecasts can be generated directly from past behaviour of the series.

This family of models includes widely used approaches such as Autoregressive (AR), Moving Average (MA), ARMA, and ARIMA models. These methods form the foundation of modern statistical time series forecasting.

Thus, autocorrelation-based forecasting shifts the focus from smoothing data to understanding and modelling dependence within the time series itself.

Assumptions Behind Autoregressive (AR) Models

Autoregressive (AR) models are forecasting techniques based on the idea that the current value of a time series depends on its past values. Before applying an AR model, certain assumptions about the behaviour of the time series must be satisfied to ensure reliable forecasting.

1. Linearity Assumption

An autoregressive model assumes a **linear relationship** between the present value and its previous observations.

For an $AR(p)$ model:

$$y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \dots + \phi_p y_{t-p} + \varepsilon_t$$

This means the current observation is expressed as a weighted sum of past values plus a random error.

2. Stationarity

The time series must be **stationary**, meaning its statistical properties remain constant over time.

This implies:

- Constant mean
- Constant variance
- Constant autocorrelation structure

If the data shows trend or seasonality, transformations such as differencing are required before fitting an AR model.

General Working Principles of the KPSS and ADF Tests

Before applying autoregressive or ARIMA models, it is necessary to determine whether a time series is stationary. Two commonly used statistical tests for checking stationarity are the Augmented Dickey–Fuller (ADF) test and the Kwiatkowski–Phillips–Schmidt–Shin (KPSS) test. Although both tests examine stationarity, they follow opposite hypotheses and complementary approaches.

1. Augmented Dickey–Fuller (ADF) Test

The ADF test is used to check whether a time series contains a **unit root**, which indicates non-stationarity.

Working Principle: The test examines whether the current value of a series depends strongly on its previous value in a way that causes persistence over time.

The ADF test estimates a regression of the form:

$$\Delta y_t = \alpha + \beta t + \gamma y_{t-1} + \sum_{i=1}^p \delta_i \Delta y_{t-i} + \varepsilon_t$$

where:

- Δy_t : Represents differenced data
- lagged differences remove autocorrelation in errors.

The key parameter tested is γ .

Hypotheses

- **Null Hypothesis (H_0):** The series has a unit root (non-stationary).
- **Alternative Hypothesis (H_1):** The series is stationary.

Decision Rule

- If the test statistic is sufficiently negative $\rightarrow$ reject $H_0 \rightarrow$ series is stationary.
- Otherwise $\rightarrow$ series is non-stationary.

Thus, the ADF test attempts to **detect non-stationarity**.

2. KPSS Test

The KPSS test checks whether a time series is stationary around a mean or trend.

Working Principle: The KPSS test decomposes a time series into:

- deterministic trend,
- random walk component,
- stationary error term.

It then evaluates whether the random walk component is significant.

Hypotheses

- **Null Hypothesis (H_0):** The series is stationary.
- **Alternative Hypothesis (H_1):** The series is non-stationary.

Decision Rule

- Small test statistic $\rightarrow$ fail to reject $H_0 \rightarrow$ stationary series.
- Large test statistic $\rightarrow$ reject $H_0 \rightarrow$ non-stationary series.

Thus, the KPSS test attempts to **confirm stationarity**.

Transformation Techniques in Time Series Analysis

In time series modelling, raw data often does not satisfy important assumptions such as constant variance or stationarity. Real-world datasets frequently show increasing fluctuations, skewness, or nonlinear growth patterns. To address these issues, data transformation techniques are applied before modelling.

Transformations help stabilize variance, improve interpretability, and make statistical models more reliable.

Need for Transformation

A transformation is required when a time series exhibits:

- Increasing or decreasing variability over time
- Nonlinear trends
- Strong skewness in data distribution
- Multiplicative seasonal patterns

For example, website traffic may grow over time, and fluctuations become larger as traffic increases. Such behaviour violates the constant variance assumption.

Common Transformation Techniques

(a) Logarithmic Transformation

$$y'_t = \log(y_t)$$

Used when variability increases proportionally with the level of the series.

(b) Square Root Transformation

$$y'_t = \sqrt{y_t}$$

Useful for moderate variance instability.

(c) Box–Cox Transformation

$$y_t^{(\lambda)} = \begin{cases} \frac{y_t^\lambda - 1}{\lambda}, & \lambda \neq 0 \\ \ln(y_t), & \lambda = 0 \end{cases}$$

A flexible and generalized transformation that automatically selects the appropriate power transformation.

Using Box–Cox Transformation and Differencing to Handle Non-Stationary Time Series

In time series analysis, many real-world datasets are **non-stationary**, meaning their statistical properties change over time.

Non-stationarity usually arises due to:

- changing variance
- long-term trend
- seasonal movement

Before applying forecasting models such as ARIMA, the time series must often be transformed into a stationary series. Two important techniques used together for this purpose are the Box–Cox transformation and differencing.

1. Two Types of Non-Stationarity

Non-stationarity generally appears in two forms:

- Non-constant variance (variance changes over time)
- Non-constant mean (trend or seasonality present)

Each problem requires a different treatment.

Problem	Solution
Changing variance	Box–Cox transformation
Changing mean	Differencing

Thus, both techniques are complementary.

2. Box–Cox Transformation: Stabilizing Variance

When fluctuations increase as the level of the series increases (common in sales or website traffic data), variance is not constant.

The Box–Cox transformation rescales the data as:

$$y_t^{(\lambda)} = \begin{cases} \frac{y_t^\lambda - 1}{\lambda}, & \lambda \neq 0 \\ \ln(y_t), & \lambda = 0 \end{cases}$$

Effect

- Compresses large values
- Expands small values
- Produces approximately constant variance

After this step, the series may still contain trend or seasonality but variability becomes stable.

3. Differencing: Removing Trend

Even after variance stabilization, the mean of the series may change over time due to trend. Differencing removes this changing mean by computing:

$$= y_t - y_{t-1}$$

This calculates the change between consecutive observations.

Effect

- Eliminates linear trend
- Stabilizes the mean
- Converts many non-stationary series into stationary ones

Seasonal differencing may also be used:

$$y_t - y_{t-s}$$

where s is the seasonal period.

Autocorrelation in Time Series

Autocorrelation is a fundamental concept in time series analysis that measures the relationship between a variable and its own past values. In simple terms, it tells us how strongly current observations depend on previous observations.

Unlike ordinary correlation, which measures the relationship between two different variables, autocorrelation measures correlation within the same time series at different time lags.

Concept of Autocorrelation

If values observed close in time tend to be similar, the series is said to exhibit autocorrelation.

For a time series y_t , autocorrelation examines the relationship between:

- y_t and y_{t-1} (lag 1)
- y_t and y_{t-2} (lag 2)
- and so on

A lag represents how many time periods we move backward.

The Autocorrelation Function (ACF) is obtained by computing autocorrelation values at different lags. General formula for autocorrelation at lag k :

$$\rho_k = \frac{\text{Autocovariance at lag } k}{\text{Variance of series}} = \frac{\sum_{t=k+1}^T (y_t - \bar{y})(y_{t-k} - \bar{y})}{\sum_{t=1}^T (y_t - \bar{y})^2}$$

Types of Autocorrelation

- **Positive Autocorrelation:** Successive values move in the same direction.
Example: High traffic tends to be followed by high traffic.
- **Negative Autocorrelation:** Successive values move in opposite directions.
Example: High traffic month followed by low traffic month.
- **No Autocorrelation:** Values occur randomly without dependence.

For the given timestep, y and y_0 , autocorrelation for different lags (0, 1, 2) are calculated below:

1. For lag 0:

Timestep	1	2	3	4	5	6	...	18	19	20
y	25	7	19	31	41	18	...	15	46	50
y_0	25	7	19	31	41	18	...	15	46	50

$$\mu = 30.5$$

$$Var = 148.95$$

$$\text{Autocovariance for lag 0} = \frac{(25 - 30.5) \times (25 - 30.5) + (7 - 30.5) \times (7 - 30.5) + \dots + (50 - 30.5) \times (50 - 30.5)}{20} = 148.95$$

$$\text{Autocorrelation for lag 0} = \frac{148.95}{148.95} = 1$$

2. For lag 1:

Timestep	1	2	3	4	5	6	...	18	19	20
y	25	7	19	31	41	18	...	15	46	50
y_0	25	7	19	31	41	18	...	15	46	50
y_1	-	25	7	19	31	41	...	38	15	46

$$\mu = 30.5$$

$$Var = 148.95$$

$$\text{Autocovariance for lag 1} = \frac{(7 - 30.5) \times (25 - 30.5) + (19 - 30.5) \times (7 - 30.5) + \dots + (50 - 30.5) \times (46 - 30.5)}{20} = 33.3125$$

$$\text{Autocorrelation for lag 1} = \frac{33.3125}{148.95} = 0.2236$$

3. For lag 2:

Timestep	1	2	3	4	5	6	...	18	19	20
y	25	7	19	31	41	18	...	15	46	50
y_0	25	7	19	31	41	18	...	15	46	50
y_1	-	25	7	19	31	41	...	38	15	46
y_2	-	-	25	7	19	31	...	51	38	15

$$\mu = 30.5$$

$$Var = 148.95$$

$$\text{Autocovariance for lag 2} = \frac{(19 - 30.5) \times (25 - 30.5) + (31 - 30.5) \times (7 - 30.5) + \dots + (15 - 30.5) \times (50 - 30.5)}{20} = -36.775$$

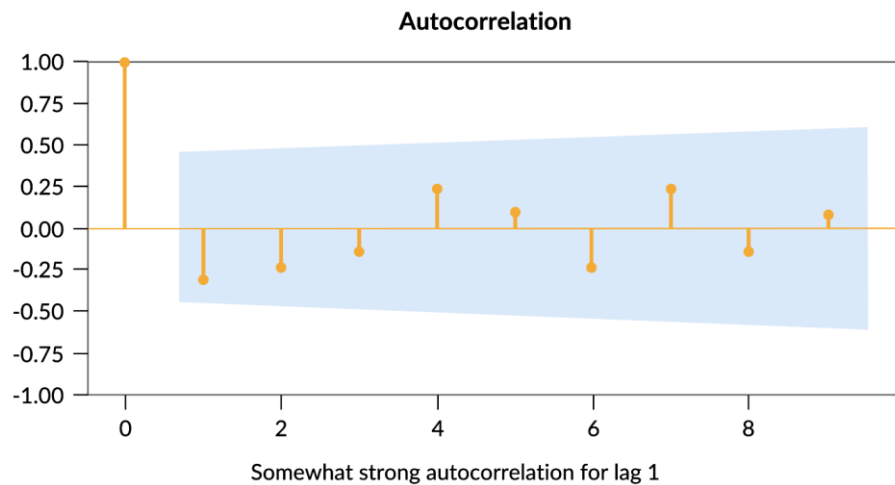
$$\text{Autocorrelation for lag 2} = \frac{-36.775}{148.95} = -0.2469$$

Visual Representation of ACF and PACF

Autocorrelation Function (ACF): Autocorrelation is commonly displayed using an ACF plot, which shows correlation values at different lags.

For any data:

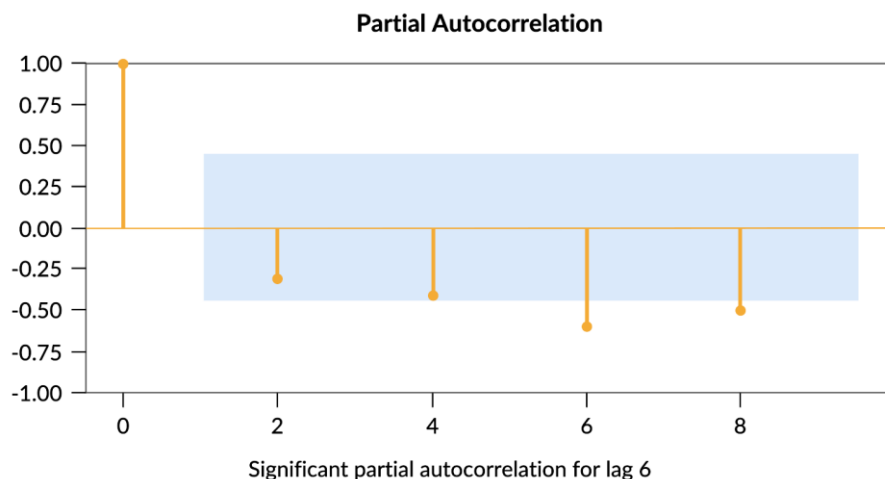
- Large spikes at early lags indicate dependence
- Gradual decay suggests trend behaviour



Partial Autocorrelation Function (PACF): It measures the correlation between a time series and a specific lag after removing the effects of intermediate lags.

It shows the pure or direct relationship between observations separated by a given lag. PACF is primarily used to determine the order of autoregressive (AR) models.

- Compute autocorrelation of a series for different lag values but skip short term dependencies
- General implementation: Skip every other value
- Less possible lag values (values less than 50% of sample size only)



The Autoregressive (AR) Model

In the realm of time-series analysis, the **Autoregressive (AR) model** stands as a fundamental pillar. At its core, an AR model is a form of linear regression that leverages a variable's own historical values to predict its future state. Unlike standard regression, which might use external factors (such as temperature or price), the “*auto*” in autoregressive signifies that the model is regressed on itself.

I. The Mathematical Foundation

The AR model operates on the premise that the current value of a series, y_t , is linearly dependent on its own previous values. This relationship is expressed as:

$$\hat{y}_t = \alpha_0 + \alpha_1 y_{t-1} + \alpha_2 y_{t-2} + \dots + \alpha_p y_{t-p}$$

where,

- $\hat{y}_t$: Predicted value at time t (dependent variable)
- $y_{t-1}, y_{t-2}, \dots, y_{t-p}$: Past observations (independent variables)
- α_0 : Intercept or constant term
- $\alpha_1, \alpha_2, \dots, \alpha_p$: Coefficients representing influence of past observations
- p : Lag order (number of past observations used)

II. Determining the Lag Order (p)

Selecting the appropriate number of past observations is critical. This process is known as determining the lag order, and the model is denoted as $AR(p)$.

The most common tool used is the Partial Autocorrelation Function (PACF) plot.

- PACF measures the correlation between a variable and its lag after removing the effects of intermediate lags.
- In a PACF plot, we look for significant spikes that stand out from background noise.
- If a strong partial autocorrelation appears at lag 3, we select:

$$p = 3$$

The regression equation for an $AR(3)$ model becomes:

$$\hat{y}_t = \alpha_0 + \alpha_1 y_{t-1} + \alpha_2 y_{t-2} + \alpha_3 y_{t-3}$$

III. Building the Model: A Step-by-Step Demonstration

Consider website traffic data observed over time steps t .

1. Data Preparation and Transformation: Raw time-series data often requires preprocessing to stabilize variance and remove trends.

- **Box-Cox Transformation:** Applied to original traffic values y to stabilize variance.
- **Differencing:** Used to make the series stationary:

$$y'_t = y_t - y_{t-1}$$

For example, at $t = 2$ the transformed value:

$$y_{\text{boxcox_diff}} = -1.27$$

2. Model Fitting: After achieving stationarity, we fit the $AR(p)$ model to estimate coefficients.

For an $AR(3)$ model applied to transformed website traffic data:

$$\begin{aligned}\alpha_0 &= 0.06 \\ \alpha_1 &= -0.71 \\ \alpha_2 &= -0.72 \\ \alpha_3 &= -0.66\end{aligned}$$

The prediction equation becomes:

$$\hat{y}_t = 0.06 - 0.71y_{t-1} - 0.72y_{t-2} - 0.66y_{t-3}$$

3. Making Predictions

The equation is used to forecast future values:

$$t = 18, 19, 20$$

For $t = 18$:

$$\hat{y}_{18} = 0.03$$

(This value is still in the transformed scale.)

4. Reconstruction (Inverse Transformation): To interpret predictions meaningfully, transformations must be reversed.

- **Step 1: Reverse Differencing (Cumulative Sum)**

$$y_t = y_{t-1} + y'_t$$

- **Step 2: Inverse Box-Cox Transformation:** (typically exponential or power inverse depending on parameter λ)

After reconstruction:

- Predicted traffic at $t = 18$: **38.98 visits**
- Predicted traffic at $t = 20$: **59.73 visits**

IV. Performance Evaluation: Model performance is evaluated by comparing predictions with actual test data.

Results

- $RMSE = 15.11$
- $MAPE = 0.63\%$

The ARMA Model

In advanced time-series analysis, the Autoregressive Moving Average (ARMA) model provides a powerful framework by merging two distinct methodologies into a single predictive engine. By simultaneously accounting for a variable's historical momentum and the lingering impact of past prediction errors, ARMA offers a more nuanced approach than either component alone.

The ARMA model is defined as the combination of:

1. Autoregressive (AR) Component

- Captures the linear relationship between the current value and its past observations.
- Represents persistence or momentum in the series.

2. Moving Average (MA) Component

- Models the current value using past prediction errors (also called shocks).
- Captures short-term random fluctuations not explained by past values.

When combined, the resulting $ARMA(p, q)$ model uses:

- p past observations
- q past prediction errors

to forecast future values.

I. The Mathematical Equation

The unified ARMA model integrates both AR and MA structures

$$\hat{y}_t = \alpha_0 + \sum_{i=1}^p \alpha_i y_{t-i} + \sum_{j=1}^q \beta_j \epsilon_{t-j}$$

where,

- $\hat{y}_t$: Predicted value at time t
- α_0 : Constant (intercept) term
- $\alpha_i y_{t-i}$: Autoregressive (AR) component
- $\beta_j \epsilon_{t-j}$: Moving Average (MA) component
- ϵ_{t-j} : Past prediction errors (white noise)
- p : AR order
- q : MA order

II. Building the Model: Website Traffic Case Study

To demonstrate the ARMA model in practice, consider website traffic data analysed using an ARMA(3,1) configuration.

This implies:

$$p = 3, q = 1$$

meaning:

- Three past observations are used
- One past prediction error is incorporated

1. Model Fitting and Estimated Coefficients

After preprocessing the data using box-cox transformation and differencing, the estimated parameters are:

- **Constant Term:** $\alpha_0 = 0.04$
- **AR Coefficients:** $\alpha_1 = -0.33, \alpha_2 = -0.58, \alpha_3 = -0.55$
- **MA Coefficient:** $\beta_1 = -0.99$

ARMA(3,1) Prediction Equation:

$$\hat{y}_t = 0.04 - 0.33y_{t-1} - 0.58y_{t-2} - 0.55y_{t-3} - 0.99\epsilon_{t-1}$$

2. Generating Predictions and Reconstruction

Predictions are generated on the transformed series for:

$$t = 18, 19, 20$$

To interpret forecasts as actual website visits, inverse transformations are applied:

- Reverse differencing (cumulative summation)
- Inverse Box-Cox transformation

Reconstructed Predictions

Time (t)	Original Traffic (y)	ARMA Predicted Traffic ($\hat{y}_t$)
18	15	31.15
19	46	39.85
20	50	54.38

(Values obtained after inverse transformation.)

III. Evaluating Model Performance: Model accuracy is assessed by comparing reconstructed predictions with actual observations.

Model Performance Results

- **RMSE** = 10.29
- **MAPE** = 0.43%

These results demonstrate that combining both AR and MA components allows the ARMA model to capture both systematic dependence and random shocks, often achieving higher predictive precision than using either method independently.

The ARIMA Model

While the Autoregressive Moving Average (ARMA) model serves as a robust tool for stationary time-series data, real-world datasets such as website traffic, often exhibit trends or changing means that violate the assumption of stationarity.

The Autoregressive Integrated Moving Average (ARIMA) model extends ARMA by incorporating a differencing mechanism, allowing it to model non-stationary data effectively. By integrating this transformation directly into its structure, ARIMA becomes one of the most widely used forecasting models in practical time-series analysis.

I. ARIMA as an Extension of ARMA

The ARIMA model can be viewed as an ARMA model combined with an integration step.

- ARMA models assume the data is already stationary.
- ARIMA models recognize that most real-world data requires preprocessing to remove trends or varying means.

The integration component (I) introduces differencing into the modelling framework, enabling forecasting on non-stationary time series with approximately constant variance.

Thus,

$$\text{ARIMA} = \text{ARMA} + \text{Differencing}$$

II. The Additional Parameter: d (Order of Differencing)

Moving from an $ARMA(p, q)$ model to an $ARIMA(p, d, q)$ model introduces a third parameter:

Model Parameters

- p : Autoregressive order (Number of past observations included)
- d : Integrated order (Number of times the series is differenced to achieve stationarity)
- q : Moving Average order (Number of past prediction errors included)

Differencing Operation

If $d = 1$, the model operates on changes between consecutive observations:

$$z_t = y_t - y_{t-1}$$

where z_t represents the differenced (stationary) series.

Example: ARIMA(1,1,1)

The prediction equation for a standard $ARIMA(1,1,1)$ model is:

$$\hat{z}_t = \alpha_1 z_{t-1} + \beta_1 \epsilon_{t-1}$$

where,

- z_t : Differenced series
- ϵ_{t-1} : Previous prediction error

III. Building the Model: Website Traffic Case Study

Using the same website traffic dataset, we apply the following configuration:

$$ARIMA(1,1,1)$$

1. General Modelling Procedure: The ARIMA workflow automatically stabilizes the time series through the following steps:

- **Step 1: Box-Cox Transformation:** Stabilizes the variance of website visits y .
- **Step 2: Integrated Differencing (d):** The model computes differences to achieve stationarity.
- **Step 3: Model Fitting:** AR and MA parameters are estimated using the differenced data

2. Model Coefficients and Predictions: The model is trained using observations up to:

$$t = 17$$

and forecasts are generated for:

$$t = 18, 19, 20$$

Forecast Results

Time (t)	Original Visits (y)	ARIMA Predicted Visits ($\hat{y}_t$)
18	15	44.01
19	46	46.51
20	50	49.15

IV. Evaluating Model Performance

The strength of ARIMA lies in its ability to address non-stationarity while preserving the predictive capability of AR and MA components.

Model Accuracy Results

- **RMSE** = 16.76
- **MAPE** = 0.65%

By formally incorporating the differencing parameter d , the **ARIMA model** provides a unified mathematical framework that:

- transforms non-stationary data,
- models temporal dependence,
- and produces reliable forecasts within a single integrated process.

This makes ARIMA one of the most practical and widely applied models in real-world time-series forecasting.

The SARIMA Model

While the Autoregressive Integrated Moving Average (ARIMA) model effectively addresses trends and non-stationarity, many real-world time-series datasets such as website traffic, also exhibit periodic fluctuations, commonly known as seasonality.

The Seasonal Autoregressive Integrated Moving Average (SARIMA) model extends ARIMA by explicitly incorporating seasonal behaviour into the forecasting framework. By adding seasonal components, the model can “remember” patterns occurring during the same period in previous cycles, improving predictive accuracy for cyclic data.

I. The Seasonal Parameters: P, D, Q , and m

A SARIMA model is written as:

$$SARIMA(p, d, q)(P, D, Q)_m$$

where,

- (p, d, q) : Represent non-seasonal components (same as ARIMA)
- $(P, D, Q)_m$: Describe the seasonal structure

Non-Seasonal Parameters

- p : Autoregressive order
- d : Order of differencing
- q : Moving average order

Seasonal Parameters

- **m : Seasonal Period** (Number of observations per seasonal cycle)

For example:

- $m = 12$: Monthly data with yearly seasonality
- $m = 7$: Daily data with weekly seasonality

- **P : Seasonal Autoregressive Order** (Number of past seasonal observations influencing the current value)
- **D : Seasonal Difference Order** (Number of seasonal differencing operations used to remove repeating seasonal trends)

Seasonal differencing is expressed as:

$$y_t - y_{t-m}$$

- **Q: Seasonal Moving Average Order** (Number of past seasonal prediction errors included in the model)

Parameter Selection

- p, d, q are typically chosen using ACF and PACF plots.
- Seasonal parameters P, D, Q are often selected using grid search, which finds the parameter combination minimizing forecasting error.

II. Building the Model: Website Traffic with Seasonality

To illustrate the SARIMA approach, we apply it to the website traffic dataset, assuming seasonal patterns exist in the observations.

1. General Modelling Procedure: The SARIMA workflow expands previous models by incorporating seasonal adjustments.

- **Step 1: Box-Cox Transformation:** Stabilizes variance in raw traffic counts y .
- **Step 2: Seasonal and Non-Seasonal Differencing:** This removes:
 - Local trends (d)
 - Seasonal patterns (D)
 - to achieve stationarity.
- **Step 3: Model Fitting:** Parameters (p, d, q) and $(P, D, Q)_m$ are estimated simultaneously.

2. Model Predictions and Reconstruction

The SARIMA model is trained using data up to:

$$t = 17$$

Forecasts are generated for:

$$t = 18, 19, 20$$

After reversing transformations (inverse differencing and inverse Box-Cox), predictions return to the original scale of website visits.

Forecast Results

Time (t)	Original Traffic (y)	SARIMA Predicted Traffic ($\hat{y}_t$)
18	15	36.96
19	46	50.56
20	50	61.16

III. Evaluating Model Performance

By incorporating seasonal parameters, SARIMA captures periodic behaviours that standard ARIMA models may overlook.

Model Accuracy Results

- **RMSE** = 16.63
- **MAPE** = 0.61%

By explicitly modelling the seasonal period m , the SARIMA model provides a sophisticated forecasting framework for datasets where cyclical timing such as the time of year, month, or day of the week, is as important as recent observations.

SARIMA therefore represents a natural progression:

$$AR \rightarrow ARMA \rightarrow ARIMA \rightarrow SARIMA$$

where each step increasing the model's ability to represent real-world temporal complexity.

